

Exp No.: 7 Endorsement Policy Configuration in Hyperledger Fabric

Aim

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

Software Requirements

- ☐ Kali Linux / Ubuntu
- ☐ Docker
- ☐ Docker Compose
- ☐ Node.js (Version 18 or later)
- ☐ npm
- ☐ Git
- ☐ Hyperledger Fabric Samples
- ☐ Hyperledger Fabric Binaries
- ☐ Hyperledger Fabric Docker Images
- ☐ Visual Studio Code (Optional)

Hardware Requirements

- ☐ Processor: Intel Core i3 or above
- ☐ RAM: Minimum 8 GB
- ☐ Storage: Minimum 20 GB Free Disk Space
- ☐ Internet Connection

Theory

An Endorsement Policy defines which organizations must approve (endorse) a transaction before it can be committed to the ledger. Hyperledger Fabric uses endorsement policies to ensure that transactions are validated according to predefined business rules.

For example:

`OR('Org1MSP.peer','Org2MSP.peer')`

means that either Org1 or Org2 can endorse the transaction.

Similarly,

`AND('Org1MSP.peer','Org2MSP.peer')`

means that both organizations must endorse the transaction before it becomes valid.

Endorsement policies improve security, trust, and transaction integrity in permissioned blockchain networks.

Procedure

Step 1: Navigate to the Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Expected Output

Creating channel 'mychannel'

Starting peer0.org1

Starting peer0.org2

Starting orderer.example.com

Network Started Successfully

Step 3: Deploy Chaincode with OR Endorsement Policy

Deploy the Asset Transfer chaincode with an endorsement policy that allows either organization to endorse transactions.

```
./network.sh deployCC \
```

```
-ccl javascript \
```

```
-ccn basic \
```

```
-ccp ../asset-transfer-basic/chaincode-javascript \
```

```
-ccep "OR('Org1MSP.peer','Org2MSP.peer')"
```

Expected Output

Packaging Chaincode

Installing Chaincode

Approving Chaincode

Committing Chaincode

Chaincode committed successfully

Step 4: Verify Chaincode Commitment

```
peer lifecycle chaincode querycommitted \
```

```
-C mychannel
```

Expected Output

Name: basic

Version: 1.0

Sequence: 1

Step 5: Initialize the Ledger

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
  
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"InitLedger","Args":[]}'
```

Expected Output

Ledger Initialized Successfully

Step 6: Query All Assets

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

Expected Output

Assets retrieved successfully.

Step 7: Stop the Network

```
./network.sh down
```

Step 8: Restart the Network

```
./network.sh up createChannel -ca
```

Step 9: Deploy Chaincode with AND Endorsement Policy

Deploy the chaincode requiring endorsements from both organizations.

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

Expected Output

Chaincode committed successfully with AND policy.

Step 10: Submit a Transaction

Invoke the chaincode transaction.

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  

```

```
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
```

Expected Output

Transaction endorsed successfully by Org1 and Org2.

Step 11: Verify the Asset

peer chaincode query \

```
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset25"]}'
```

Expected Output

```
{  
  "ID":"asset25",  
  "Owner":"Arun",  
  "Color":"Blue",  
  "Size":10,  
  
  "AppraisedValue":1200  
}
```

Step 12: Stop the Network

./network.sh down

Flow Diagram

Start Fabric Network

|

▼

Deploy Chaincode

|

▼

Configure Endorsement Policy

|

▼

Submit Transaction Proposal

|

▼

Collect Endorsements

|

▼

Validate Transaction

|



Commit Transaction

|



Update Ledger

|



Query Ledger

|



Stop Network

Architecture Diagram

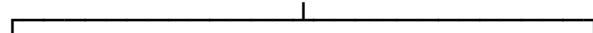
Client Application

|



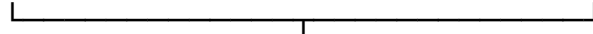
Transaction Proposal

|



Peer Org1 Peer Org2
(Endorsement) (Endorsement)

|



Orderer

|



Blockchain Ledger

Sample Output

Fabric Network Started Successfully

Chaincode Installed Successfully

OR Policy Configured Successfully

AND Policy Configured Successfully

Transaction Endorsed Successfully

Ledger Updated Successfully

```

n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME2_TLS_ROOTCERT_FILE" \
-c '{ "function": "InitLedger", "Args": [] }' \
--waitForEvent
2026-08-20 19:02:58.185 IST 0001 INFO [chaincodeCmd] ClientWait → txid [81df3219ce607663e4259def2436487f7287bc1af0e07520aaa20ae91df4383] committed with status (VALID) at localhost:7051
2026-08-20 19:02:58.186 IST 0002 INFO [chaincodeCmd] ClientWait → txid [81df3219ce607663e4259def2436487f7287bc1af0e07520aaa20ae91df4383] committed with status (VALID) at localhost:9051
2026-08-20 19:02:58.187 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
C privatechannel \
n basic \
-c '{ "function": "GetAllAssets", "Args": [] }'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C privatechannel \
n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME2_TLS_ROOTCERT_FILE" \
-c '{ "function": "CreateAsset", "Args": ["private01", "Black", "20", "PrivateUser", "5000"] }' \
--waitForEvent
2026-08-20 19:03:17.548 IST 0001 INFO [chaincodeCmd] ClientWait → txid [58b391216de45a58569578c4f2e88c2f921e75f2ceecd4b855c07c56e72a897] committed with status (VALID) at localhost:7051
2026-08-20 19:03:17.548 IST 0002 INFO [chaincodeCmd] ClientWait → txid [58b391216de45a58569578c4f2e88c2f921e75f2ceecd4b855c07c56e72a897] committed with status (VALID) at localhost:9051
2026-08-20 19:03:17.549 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload:{"ID":"private01","Color":"Black","Size":20,"Owner":"PrivateUser","AppraisedValue":5000}

What you should capture for your lab record
The four best screenshots are:
1. The initial ledger state (before the first transaction)
2. The ledger state after the first transaction (CreateAsset)
3. The ledger state after the second transaction (CreateAsset)
4. The ledger state after the third transaction (CreateAsset)

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
C mychannel \
n basic \
-c '{ "function": "ReadAsset", "Args": ["private01"] }'
[{"AppraisedValue":5000,"Color":"Black","ID":"private01","Owner":"PrivateUser","Size":20}]

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C privatechannel \
n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME2_TLS_ROOTCERT_FILE" \
-c '{ "function": "InitLedger", "Args": [] }' \
--waitForEvent
2026-08-20 19:02:58.185 IST 0001 INFO [chaincodeCmd] ClientWait → txid [81df3219ce607663e4259def2436487f7287bc1af0e07520aaa20ae91df4383] committed with status (VALID) at localhost:7051
2026-08-20 19:02:58.186 IST 0002 INFO [chaincodeCmd] ClientWait → txid [81df3219ce607663e4259def2436487f7287bc1af0e07520aaa20ae91df4383] committed with status (VALID) at localhost:9051
2026-08-20 19:02:58.187 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
C privatechannel \
n basic \
-c '{ "function": "GetAllAssets", "Args": [] }'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]

(tharun@kali): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C privatechannel \
n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$HOME_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$HOME2_TLS_ROOTCERT_FILE" \
-c '{ "function": "CreateAsset", "Args": ["private01", "Black", "20", "PrivateUser", "5000"] }' \
--waitForEvent
2026-08-20 19:03:17.548 IST 0001 INFO [chaincodeCmd] ClientWait → txid [58b391216de45a58569578c4f2e88c2f921e75f2ceecd4b855c07c56e72a897] committed with status (VALID) at localhost:7051
2026-08-20 19:03:17.548 IST 0002 INFO [chaincodeCmd] ClientWait → txid [58b391216de45a58569578c4f2e88c2f921e75f2ceecd4b855c07c56e72a897] committed with status (VALID) at localhost:9051
2026-08-20 19:03:17.549 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload:{"ID":"private01","Color":"Black","Size":20,"Owner":"PrivateUser","AppraisedValue":5000}

```

```

Chaincode definition approved on peer0.org2 on channel 'privatechannel'
Using organization 1
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'privatechannel' ...
Attempting to check the commit readiness of the chaincode definition on peer0.org1. Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID privatechannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'privatechannel'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'privatechannel' ...
Attempting to check the commit readiness of the chaincode definition on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID privatechannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'privatechannel'
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7058 --ordererTLSHostnameOverride orderer.example.com --tls -cafile /home/tharun/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID privatechannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /home/tharun/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tlsca1.example.com-cert.pem --peerAddresses localhost:7051 --tlsRootCertFiles /home/tharun/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem --version 1 --sequence 1
+ res=0
2026-08-20 19:02:24.167 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [0d7de0562e18eb6865bda20db5e5875b35fc93e31c447fff8186d8c6967cab4e] committed with status (VALID) at localhost:7051
2026-08-20 19:02:24.167 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [0d7de0562e18eb6865bda20db5e5875b35fc93e31c447fff8186d8c6967cab4e] committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'privatechannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'privatechannel' ...
Attempting to Query committed status on peer0.org1. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID privatechannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'privatechannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'privatechannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'privatechannel' ...
Attempting to Query committed status on peer0.org2. Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID privatechannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'privatechannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'privatechannel'

```

```

tharun@kali:~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help

[tharun@kali]~/fabric-dev/fabric-samples/test-network
$ export PATH=$PATH:/home/fabric-dev/fabric-samples/bin
$ export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config

$ export CORE_PEER_TLS_ENABLED=true
$ export CORE_PEER_LOCALMSPID=Org1MSP
$ export CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem
$ export CORE_PEER_ADDRESS=localhost:7051
$ export CORE_PEER_PROFILE_PATH=$PWD/organizations/peerOrganizations/org1.example.com/users/AdminOrg1.example.com/msp

$ export ORG2_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem
$ export ORDERER_CA=$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlsacerts/tlsca.example.com-cert.pem

[tharun@kali]~/fabric-dev/fabric-samples/test-network
$ peer channel list
2026-08-20 19:01:06.196 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Channels peers has joined:
mychannel

[tharun@kali]~/fabric-dev/fabric-samples/test-network
$ ./network.sh createChannel -c privatechannel
Using docker and docker-compose
Creating channel 'privatechannel'
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb'
Network Running Already
Using docker and docker-compose
Generating channel genesis block 'privatechannel.block'
Using organization 1
/home/tharun/fabric-dev/fabric-samples/test-network/.../bin/configtxgen
+ '[' 0 -eq 1 ']'
+ configtxgen -profile ChannelUsingRaft -outputBlock ./channel-artifacts/privatechannel.block -channelID privatechannel
2026-08-20 19:01:13.443 IST 0001 INFO [common.tools.configtxgen] main -> Loading configuration
2026-08-20 19:01:13.446 IST 0002 INFO [common.tools.configtxgen.localconfig] completeInitialization -> Orderer type: etcdraft
2026-08-20 19:01:13.446 IST 0003 INFO [common.tools.configtxgen.localconfig] completeInitialization -> Orderer.Etcdraft.Options unset, setting to tickInterval:"500ms" electionTick:10 heartbeatTick:1 maxInflightBlocks:5 snapshotIntervalSize:16777216
2026-08-20 19:01:13.446 IST 0004 INFO [common.tools.configtxgen.localconfig] load -> Loaded configuration: /home/tharun/fabric-dev/fabric-samples/test-network/configtx/configtx.yaml
2026-08-20 19:01:13.447 IST 0005 INFO [common.tools.configtxgen] doOutputBlock -> Generating genesis block
2026-08-20 19:01:13.447 IST 0006 INFO [common.tools.configtxgen] doOutputBlock -> Creating application channel genesis block
2026-08-20 19:01:13.447 IST 0007 INFO [common.tools.configtxgen] doOutputBlock -> Writing genesis block
+ res=0
Creating channel privatechannel
Adding peers:
+ scripts/orderer.sh privatechannel
+ '[' 0 -eq 1 ']'
+ res=0
2026-08-20 19:00:47.512 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-08-20 19:00:47.528 IST 0002 INFO [channelCmd] update -> Successfully submitted channel update
Status: 201
{
  "name": "privatechannel",
  "url": "/participation/v1/channels/privatechannel",
  "contentType": "application/json"
}

```

Result

The endorsement policies were successfully configured in the Hyperledger Fabric network. Transactions were validated according to the specified endorsement rules, demonstrating how Hyperledger Fabric ensures trust, integrity, and secure transaction processing through endorsement policies.